

Analisi Tecnica Dettagliata — Playbook di Sviluppo · Progetto SFT

- Analisi Tecnica Dettagliata — Playbook di Sviluppo
 - Progetto SFT — Nuovo sito + gestionale come motore unico
 - METODO DI LAVORO (vale per ogni passo)
 - FASE 0 — PREPARAZIONE
 - FASE 1 — DATABASE, FUNZIONI, API DI LETTURA
 - FASE 2 — GESTIONALE (MAUI Blazor) — l'operatore gestisce tutto qui
 - FASE 3 — NUOVO SITO WEB (SEO-first, 5 lingue)
 - FASE 4 — PAGAMENTI ONLINE + PROMEMORIA (per-azienda)
 - FASE 5 — MIGRAZIONE DATI & GO-LIVE
 - APPENDICE — Checklist trasversali
 - AGGIORNAMENTO DECISIONI — scelte di dettaglio (2026-07-03)

Analisi Tecnica Dettagliata — Playbook di Sviluppo

Progetto SFT — Nuovo sito + gestionale come motore unico

 **USO INTERNO (Adriano + AI).** Questo è il documento operativo che seguiamo per realizzare il lavoro, a passi sequenziali. Base funzionale: [Analisi_Preliminare_Sito_Web_SFT_v2.md](#) (v2.1). **Versione:** 1.0 • **Data:** 20 Giugno 2026

METODO DI LAVORO (vale per ogni passo)

Ambienti — sempre locale prima, produzione dopo i test. 1. Si sviluppa e si testa in **locale**: DB su Docker `postgres_db` (PostgreSQL 17.5), gestionale MAUI in debug, sito in ambiente di sviluppo. 2. Solo quando i test del passo sono **verdi**, si porta in **produzione** (Hetzner/Supabase), con backup preventivo. 3. Nessuna modifica diretta in produzione senza prima averla validata in locale.

Regole architetturali (non negoziabili). - **DB-First:** zero SQL inline nei servizi. Ogni operazione passa da **funzioni PL/pgSQL**. Dopo ogni funzione nuova/modificata → aggiornare `Documents/Funzioni_DB.md`. - **Script versionati:** ogni modifica DB è uno script numerato in `SqlScripts/` (deploy: `docker exec -i postgres_db psql -U postgres -d gestione_viaggi < SqlScripts/xxx.sql`). - **Multi-tenant:** `azienda_id` su tutte le tabelle di dominio + policy RLS coerenti con `superadmin_bypass_all`. - **Dapper:** `DefaultTypeMap.MatchNamesWithUnderscores = true`; usare cast espliciti `::DATE` / `::VARCHAR` quando si passano `DateTime?` / `string?` alle funzioni (evita errori 42883 di overload). - **Servizi** registrati Singleton in `MauiProgram.cs`; pattern Dapper + `IDatabaseService` + `ILogger`. - **Componenti già in casa:** Blazored.TextEditor (rich text), SixLabors.ImageSharp (immagini), SkiaSharp (mappe), QuestPDF (PDF), MudBlazor 8.15 (UI).

Vincoli di sviluppo OBBLIGATORI (MUST — inderogabili). - **Componenti condivisi (riuso prima di tutto).** I componenti UI vivono separati dal codice principale in `Components/Shared/` e sono catalogati in `Documents/ComponentiShared.md` (es. `EnterpriseDataGrid`, `EnterpriseActionsColumn`, `EnterprisePager`, `BaseEntitySelect`, `CountrySelect`...). Regola: **se il componente esiste, si riusa**; se ne serve uno nuovo, **si scrive una sola volta** come componente condiviso e si **aggiorna** `ComponentiShared.md`. Vietato duplicare UI inline. - **Controlli/validazioni centralizzati.** I check sono **unici e centralizzati** in `Validation/` (Core / Syntax / Semantic / Business / Extensions / Database) come da `Documents/Gestione_check.md`, integrati con **FluentValidation** e messaggi ITA centralizzati (`DbErrorTranslator` per gli errori PostgreSQL). Regola: **se il validatore esiste, si riusa**; se ne serve uno nuovo, **si scrive una sola volta** nel posto giusto e si **aggiorna** `Gestione_check.md`. Vietate validazioni sparse/duplicate. - **Regole UI delle form di edit (sempre):** 1. **Set-focus** automatico sul **primo campo** della form. 2. **Maiuscolo forzato** sui campi alfanumerici **dei dati gestionali/anagrafici** (i minuscoli non sono accettati). ⚠ **ECCEZIONE:** i **campi editoriali dei contenuti web** (titolo, sottotitolo, descrizioni, luoghi, info...) e **tutti i campi RichText NON** vanno forzati in maiuscolo: conservano maiuscole/minuscole e la **formattazione** decisa dall'operatore, perché vengono mostrati ai clienti finali sulle pagine web in lettura. 3. **Tabulazione tra i campi** gestita con l'**helper JS dedicato già presente** e usato ovunque. - Queste regole **e quelle già documentate** nei due file sopra sono un **MUST**: vanno seguite senza eccezioni in ogni passo (in particolare nella Fase 2 – gestionale).

Definizione di “fatto” per ogni passo: codice + DB aggiornati in locale · funzioni documentate · test del passo superati · (a fine fase) deploy in produzione verificato.

Stack tecnologico — DECISIONI CHIUSE (20/6/2026). - **Database:** Supabase (Postgres gestito) — si prosegue su questo. Il sito **non** usa un microservizio API separato: legge i dati **lato server in Next.js** (Server Components / Route Handlers con `@supabase/ssr`), esponendo

solo i contenuti pubblicati. Sicurezza: policy **RLS** per il ruolo pubblico/anon che consentono la lettura dei soli contenuti web pubblicati e **negano** ogni accesso alle tabelle operative/contabili. Usare il **connection pooler Supabase** (PgBouncer, transaction mode), importante con il serverless. - **Frontend: Next.js (App Router, React)** — SEO-first con **ISR** (pagine tour statiche + rigenerazione in background) e **revalidation on-demand**: quando l'operatore preme "Pubblica" nel gestionale, un endpoint/webhook sicuro rigenera **subito** la pagina interessata. Sinergia con Supabase ([@supabase/ssr](#)) e Stripe. (Il sito pubblico non ha login utente: [@supabase/ssr](#) serve per il fetch dati lato server, non per autenticazione.) - **Hosting del sito — CONFERMATO: Hetzner (Coolify) + Cloudflare** (coerente col costo ~10 €/mese comunicato a SFT e con UE/GDPR). (Vercel scartato: era un refuso da altro progetto.) - **Mappe — fornitore: GEOAPIFY Static Maps API (DECISO 22/6 — GRATIS)**. Mantiene la promessa "mappe gratuite" fatta a SFT e protegge la traccia. **Approccio: immagine STATICA** (non mappa interattiva a tiles): una mappa interattiva manderebbe le coordinate della traccia al browser → riestraibili; l'immagine statica le "cuoce" nei pixel. **Flusso (tutto lato gestionale C#, dove il GPX viene caricato — NON una Server Action Next.js)**: decimazione traccia (**Douglas-Peucker**, es. NetTopologySuite) → bbox + margine → chiamata **Geoapify Static Maps** (stile `osm-bright` / `osm-liberty`, colori personalizzati verso palette earthy/outdoor; linea percorso + marker tappe curati) con la polyline semplificata → **WebP** (ImageSharp) → **upload su Supabase Storage** (bucket pubblico `tour-maps`) → si salva l'URL pubblico in `web_tour_mappa`. Il sito Next.js renderizza solo quell'URL (CDN, **zero chiamate API per visita**). Geoapify (OSM/ODbL) **consente** storage/ridistribuzione con attribuzione © **OpenStreetMap contributors**; free tier sufficiente (si genera una volta sola, ~67 tour). La traccia/GPX **non raggiunge mai il browser**. Mapbox/MapTiler esclusi (vietano lo storage server-side). **Stadia (stile terrain Stamen)** resta un possibile **upgrade premium a pagamento** da proporre in futuro. - **Pagamenti: Stripe con chiavi/credenziali per azienda** (no Connect: ogni azienda ha il proprio account, incassa direttamente). Chiavi **segrete cifrate** per-azienda (come `ana_aziende_smtp.password_enc`), usate **solo lato server; webhook secret per-azienda** con verifica firma. Per acconti/saldi nei promemoria → **Stripe Checkout / Payment Links** (hosted: PCI minimo, nessuna UI carte da costruire). - **Token iscrizione (verificato)**: l'app iscrizioni (Flask su **Hetzner**) è multi-azienda via prefisso URL `/<azienda_id>-<token>/`; `AZIENDA_ID` e `FLASK_SECRET_KEY` stanno nel `.env` **sul server Hetzner**, non in DB. → Per generare il link "Iscriviti" dal gestionale: **consigliato** salvare il token per-azienda in `ana_aziende` (DB condiviso) così gestionale e Flask lo usano entrambi (in alternativa, un piccolo endpoint esposto dall'app iscrizioni). *Da rifinire leggendo la logica esatta in `app.py` (~riga 353).*

FASE 0 — PREPARAZIONE

Passo 0.1 — Ambienti e baseline

- **Obiettivo**: partire da una base pulita e riproducibile.
- **Attività**: branch/worktree git dedicati; snapshot/backup del DB locale e di produzione; verifica versioni; allineamento `Funzioni_DB.md` allo stato attuale.
- **Verifica**: ambiente locale che parte, DB raggiungibile, backup ripristinabile.

Passo 0.2 — Chiusura decisioni tecniche

- **Stato: TUTTE CHIUSE**. Hosting = **Hetzner + Cloudflare**; mappe = **Geoapify Static Maps (gratis)**; DB = **Supabase**; frontend = **Next.js/ISR**; **Stripe chiavi per-azienda**; token iscrizione su Hetzner.
- Unico residuo operativo: verifica fine della logica token in `app.py` (~riga 353) quando si implementa il link "Iscriviti".

FASE 1 — DATABASE, FUNZIONI, API DI LETTURA

Passo 1.1 — Schema nuove tabelle (script SQL)

- **Obiettivo**: creare lo scheletro dati completo (incluse le predisposizioni).
-  **Spec dettagliata campo-per-campo** (modifiche a tabelle esistenti + tutte le nuove tabelle): vedi `Dettaglio_Tabelle_DB.md` (documento companion).
- **Attività (script numerati in `SqlScripts/`)**:
 - `web_tour_contenuti` (1:1 `ana_viaggi`): sottotitolo, descrizione_html, difficoltà, durata_testo, luoghi_visitati, info_pernottamento/pasti/equipaggiamento/altre_info_html, slug, meta_title, meta_description, stato_publicazione, ordine, web_categoria_sport_fk.
 - `web_tour_itinerario` (N): giorno_numero, titolo_giornata, ordine.
 - `web_tour_itinerario_passaggi` (N): testo_html, immagine/bytea o url, didascalia, ordine.
 - `web_tour_immagini` (N): tipo (principale/galleria), immagine, alt_text, titolo, ordine.
 - `web_tour_mappa` (1:1): gpx (bytea), bbox (min/max lat/lon), immagine_render, provider, parametri_render, data_generazione.
 - `web_traduzioni` (per-campo): entità, campo, lingua, testo_tradotto, tradotto_auto, revisionato, obsoleto, data.
 - `web_categorie_sport` + tabella di mappatura da `ana_tipo_viaggi`.
 - `web_newsletter_iscritti`, `web_newsletter_invii` (+ righe destinatari), `web_newsletter_soppressioni`.

- Estensione `ana_aziende_smt` per tipo ESP (provider, api_key cifrata, dominio mittente) ◦ `ana_aziende_esp`.
 - `ana_clienti`: aggiunta `consenso_marketing` (bool) + `consenso_marketing_data` + fonte.
 - `web_aziende_funzioni` (toggle per-azienda: recensioni, pagamenti_online, blog, newsletter_esp + parametri jsonb).
 - **Predisposizione** (creare ma non usare subito): `web_pagamenti_regole`, `web_pagamenti_reminder_regole`, `web_pagamenti_transazioni`, `web_pagamenti_reminder_log`, `web_blog_articoli`.
 - Tutte con `azienda_id`, audit `created/updated`, indici e **policy RLS**.
- **Verifica:** deploy su DB locale senza errori; vincoli e RLS testati; rollback script pronto.

Passo 1.2 — Funzioni PL/pgSQL (CRUD)

- **Obiettivo:** esporre ogni operazione come funzione (DB-first).
- **Attività:** funzioni `get/list/insert/update/delete` per ogni entità del passo 1.1; funzioni “di servizio” (es. unione+dedup destinatari newsletter; calcolo prezzo “da” minimo; lista tour pubblicati per azienda/lingua).
- **Deliverable:** funzioni + aggiornamento `Funzioni_DB.md`.
- **Verifica:** chiamata di ogni funzione con dati di test; controllo multi-azienda (un’azienda non vede l’altra).

Passo 1.3 — Strato dati pubblico (lettura) per il sito

- **Obiettivo:** esporre al sito **solo i contenuti pubblicati**, senza microservizio separato.
- **Attività:** **policy RLS** per il ruolo pubblico/anon che consentono la lettura dei soli `stato_publicazione = pubblicato` (per azienda e lingua) e **negano** ogni accesso alle tabelle operative/contabili; viste/funzioni di lettura per lista tour, dettaglio, itinerario, immagini, mappa, categorie, prezzi/date, traduzioni. Il **sito legge lato server in Next.js** (`@supabase/ssr`); chiavi privilegiate **mai** nel browser. Connection pooler Supabase.
- **Verifica:** un contenuto in bozza **non** è leggibile; il ruolo pubblico **non** vede clienti/contabilità; risposte multilingua corrette.

FASE 2 — GESTIONALE (MAUI Blazor) — l’operatore gestisce tutto qui

In tutta la Fase 2 valgono i **Vincoli MUST del Metodo**: riuso/creazione di **componenti condivisi** (`ComponentiShared.md`), **validazioni centralizzate** (`Gestione_check.md`), e **regole UI** delle form (set-focus sul primo campo, **maiuscolo forzato** sui campi alfanumerici, **tabulazione** via helper JS dedicato).

Passo 2.1 — Strato dati/servizi

- Modelli + servizi Dapper (Singleton) per le nuove tabelle, sopra le funzioni del passo 1.2. **Verifica:** CRUD da codice end-to-end su DB locale.

Passo 2.2 — Scheda “Contenuti Web” nel viaggio

- Campi editoriali + **rich text** (Blazored.TextEditor) per descrizioni e info; campi **SEO** (slug, meta); **stato pubblicazione**. **Verifica:** salva/rilegge, slug univoco, anteprima coerente.

Passo 2.3 — Itinerario giorno-per-giorno (annidato, drag&drop)

- Giornate → N passaggi (testo rich + immagine + didascalia); riordino drag&drop (MudBlazor) a entrambi i livelli. **Verifica:** ordine persistito, riordino stabile.

Passo 2.4 — Galleria immagini

- Upload multiplo, **resize/ottimizzazione (ImageSharp)**, alt/titolo, ordinamento, foto principale. **Verifica:** immagini ridimensionate e ordinate, alt salvati.

Passo 2.5 — Mappatura categoria “Sport”

- UI di mappatura `ana_tipo_viaggi` → `web_categorie_sport` (+ etichetta web). **Verifica:** ogni tipo tecnico mappato a una categoria.

Passo 2.6 — GPX → mappa statica (generata al “Pubblica”, lato gestionale)

- Upload GPX nel gestionale → parse → **decimazione (Douglas-Peucker)** → **bbox** (+margine) → chiamata **Geoapify Static Maps** (stile `osm-bright` / `osm-liberty` con colori personalizzati, polyline semplificata) → **WebP** (ImageSharp) → **upload su**

Supabase Storage (bucket pubblico `tour-maps`) → si salva l'**URL pubblico** + bbox + parametri in `web_tour_mappa`. Generazione **una sola volta** al publish; il sito serve l'immagine via CDN (zero API/visita). Attribuzione © **OpenStreetMap contributors** sulla mappa. **Verifica:** mappa centrata su più GPX reali; traccia semplificata; **nessun dato vettoriale/GPX raggiunge il browser**; URL pubblico valido.

Passo 2.7 — Traduzione automatica (Claude API)

- Servizio di traduzione: pulsante “Traduci” → chiamata API → salvataggio **per-campo** in `web_traduzioni` con flag auto/revisionato; modifica dell'IT marca le traduzioni **obsolete**; UI di revisione. **Verifica:** 5 lingue prodotte, flag obsolescenza funzionante, nomi propri non tradotti.

Passo 2.8 — Newsletter

- Elenco iscritti (filtri), composizione (editor), **unione+dedup** clienti+iscritti per azienda, esclusione soppressioni, invio via **SMTP/ESP per-azienda**, gestione disiscrizione (token), storico invii. **Verifica:** dedup corretto, invio di prova, link disiscrizione attivo.

Passo 2.9 — Configurazione per-azienda

- UI **toggle funzioni** (`web_aziende_funzioni`); configurazione **ESP**; UI **predisposta** per regole di pagamento (attiva in Fase 4). **Verifica:** flag riflessi nel comportamento (es. recensioni on/off).

Passo 2.10 — Anteprima / Pubblica / Clona

- Anteprima di come apparirà sul sito; “Pubblica” (cambia stato); “Clona” esteso ai contenuti web. **Verifica:** clona copia itinerario+gallerie; pubblica rende visibile via API.

FASE 3 — NUOVO SITO WEB (SEO-first, 5 lingue)

Passo 3.1 — Fondamenta (Next.js App Router)

- Setup **Next.js App Router**; design system “moderno” (accattivante/fidelizzante, come richiesto da SFT); **i18n 5 lingue + hreflang**; infrastruttura SEO (sitemap, Schema.org TouristTrip/Offer/AggregateRating/BreadcrumbList, meta/OG/Twitter); lettura dati **lato server** (`@supabase/ssr` , Passo 1.3); **ISR** sulle pagine tour + **revalidation on-demand** all'evento “Pubblica” del gestionale; deploy su **Hetzner (Coolify) + Cloudflare** (*da confermare vs Vercel*). **Verifica:** pagina multilingua indicizzabile, hreflang corretti, rigenerazione immediata su publish.

Passo 3.2 — Home

- Hero, tour in evidenza, prossime partenze, filtri rapidi. **Verifica:** dati reali dall'API.

Passo 3.3 — Liste per categoria + filtri

- Filtri (mezzo, destinazione, difficoltà, durata, prezzo, **data/disponibilità**), prezzo “da” automatico, badge dinamici (Ultimi posti/Novità/Sold-out). **Verifica:** filtri combinati corretti.

Passo 3.4 — Dettaglio tour

- Itinerario giorno-per-giorno, gallery+lightbox, **mappa OSM**, scheda tecnica, prezzi/date, **CTA “Iscriviti”** (link generato verso l'app esistente), PDF programma (QuestPDF), FAQ, condivisione. **Verifica:** link iscrizione con azienda+viaggio corretti.

Passo 3.5 — Pagine statiche e conversione

- Chi siamo, gallery/video, contatti (form + WhatsApp + social), **iscrizione newsletter (double opt-in)**, privacy/cookie, **cookie consent GDPR**. **Verifica:** opt-in scrive in `web_newsletter_iscritti`.

Passo 3.6 — Calendario partenze. Verifica: date reali, link al tour.

Passo 3.7 — Recensioni (toggle per-azienda)

- Integrazione Google/TripAdvisor o gestione interna, mostrata solo se `recensioni = ON`. **Verifica:** on/off per azienda.

Passo 3.8 — Performance / SEO / accessibilità

- WebP/AVIF, lazy-load, Core Web Vitals, Lighthouse, viewport corretto (zoom abilitato), alt text. **Verifica:** punteggi Lighthouse target raggiunti.

FASE 4 — PAGAMENTI ONLINE + PROMEMORIA (per-azienda)

Passo 4.1 — Integrazione Stripe (chiavi per-azienda)

- **Chiavi/credenziali Stripe per-azienda** (no Connect): segrete **cifrate** (come `ana_aziende_smt`), usate **solo lato server**; **webhook secret per-azienda** con verifica firma. Per acconti/saldi → **Stripe Checkout / Payment Links** (hosted, PCI minimo). Registrazione transazioni in `web_pagamenti_transazioni`. **Verifica:** flussi sandbox per azienda, webhook idempotenti e con firma valida.

Passo 4.2 — Motore regole di pagamento

- `web_pagamenti_regole` per-azienda: acconto/saldo/soluzione unica + scadenze; calcolo importi e date a partire dalla prenotazione/partenza; UI config nel gestionale. **Verifica:** calcoli corretti su casi-tipo per azienda.

Passo 4.3 — Promemoria e solleciti

- **Job schedulato giornaliero** che rileva scadenze/ritardi; template **multilingua** distinti (promemoria / scaduto); invio al cliente con **CCN al tour operator** (indirizzo scelto da `ana_aziende_email`); `web_pagamenti_reminder_log` anti-duplicati. **Verifica:** invii corretti per data, nessun doppio, CCN presente.

Passo 4.4 — Integrazione contabile automatica dell'incasso (il "grande plus")

- **Obiettivo:** ogni pagamento Stripe confermato **alimenta automaticamente la contabilità già presente** nel gestionale — non solo il singolo **cliente** e il **viaggio**, ma la **contabilità generale**.
- **Come:** alla conferma del webhook (`web_pagamenti_transazioni` → `pagato`), una funzione PL/pgSQL:
 1. **crea/abbina la controparte** del cliente al **PRIMO incasso** (controparte `is_cliente` ; il link si memorizza in `ana_clienti.controparte_fk` e viene riutilizzato al saldo, evitando duplicati) — *decisione presa: creazione al primo incasso*;
 2. genera la **fattura FV** (acconto o saldo) del **ciclo ATTIVO**, con IVA/bollo secondo il **regime** (SFT = **forfettario**), **link a viaggio/data**, importo = importo Stripe, **EUR**;
 3. genera l'**incasso IN** che **salda** quella **FV** (`transazione_fattura_fk` → la FV); i **trigger esistenti** (`trg_aggiorna_stato_pagamento`) aggiornano lo **stato** (PARZIALMENTE_PAGATO/PAGATO).
- **Effetto a cascata (gratis, via le view esistenti):** `vw_partitario_clienti` , `vw_scadenzario` , `vw_margini_viaggi` e contabilità generale si aggiornano da soli. *Nessuna modifica allo schema di `mov_transazioni` : si riutilizzano tabelle, trigger e view.*
- **Commissione Stripe:** registrabile come **costo** (movimento passivo/riga), così netto e **margini** sono reali.
- **Idempotenza:** FV/IN create **una sola volta** per pagamento → link 1:1 su `web_pagamenti_transazioni` (anti-doppioni sui retry del webhook).
- **Verifica:** un pagamento sandbox genera **1 FV + 1 IN** corrette, stato fattura aggiornato, riflesso in partitario/scadenzario/margini. (Nomi colonne di `mov_transazioni` da riconfermare sul DB live.)

Passo 4.5 — Fattura automatica + copia di cortesia al cliente (decisione presa)

- **Obiettivo:** al ricevimento del pagamento, **emettere la fattura** (acconto e saldo) e **inviarla subito** al cliente.
- **Come:** generata la **FV** (Passo 4.4), si produce un **PDF di cortesia** della fattura (**QuestPDF**, già nel progetto) e lo si **invia via email** al cliente (CCN operatore opzionale), con numerazione progressiva coerente.
- **⚠ Fattura elettronica (IT):** la copia PDF è una **cortesia**; la **fattura legale è elettronica** e passa da **SdI**. Il gestionale ha già la **numerazione FatturaPA** (`ana_fatturapa_progressivi` , `mov_contattori_protocollo_iva`): la e-fattura si **innesta sul flusso esistente**. *Da confermare: trasmissione a SdI diretta o via intermediario.*
- **Verifica:** dopo il pagamento il cliente riceve la copia PDF; numerazione fattura progressiva e corretta; e-fattura allineata al flusso esistente.

FASE 5 — MIGRAZIONE DATI & GO-LIVE

Passo 5.1 — Recupero e import contenuti dall'attuale sito

- **Estrazione** dall'attuale piattaforma del sito di: contenuti tour, **itinerari giorno-per-giorno**, immagini (principale + gallerie), testi info. Mapping verso le nuove tabelle e caricamento (67 tour). **Verifica:** confronto a campione tra vecchio sito e nuovi dati; nessuna perdita.

Passo 5.2 — Import newsletter

- Import lista email dall'attuale sito → `web_newsletter_iscritti`; **matching/dedup** con `ana_clienti`; registrazione consensi. **Verifica:** nessun duplicato; conteggi coerenti.

Passo 5.3 — Traduzioni iniziali

- Traduzione batch dei 67 tour × 4 lingue; **revisione** dei testi. **Verifica:** spot-check qualità per lingua.

Passo 5.4 — SEO migration

- **Redirect 301** vecchi URL → nuovi, sitemap multilingua, hreflang, registrazione Search Console. **Verifica:** vecchie URL reindirizzano correttamente.

Passo 5.5 — Go-live

- Cutover DNS, smoke test su tutte le pagine/flussi, monitoring; **spegnimento del vecchio sito** solo dopo conferma. **Verifica:** sito nuovo stabile in produzione; iscrizioni e newsletter operative.

APPENDICE — Checklist trasversali

Checklist deploy locale → **produzione (per ogni fase):** 1. Test locali verdi. 2. Backup DB produzione. 3. Applicare script SQL numerati in ordine. 4. Deploy codice (gestionale/sito/API). 5. Smoke test in produzione. 6. Aggiornare `Funzioni_DB.md` e changelog.

Checklist test ricorrenti: multi-azienda (isolamento dati) · RLS · contenuti in bozza non pubblici · 5 lingue + hreflang · invio/dedup/disiscrizione newsletter · mappa centrata + niente GPX scaricabile · pagamenti sandbox · promemoria senza doppioni.

Documentazione da tenere aggiornata: `Funzioni_DB.md` (funzioni DB), script in `SqlScripts/`, manuale utente per le nuove funzioni del gestionale.

AGGIORNAMENTO DECISIONI — scelte di dettaglio (2026-07-03)

Rispetto alla stesura iniziale, in fase di progettazione esecutiva e di avvio implementazione sono state prese scelte più fini. Questa sezione le consolida; prevale su quanto sopra dove diverge.

A. Perimetro e rilascio

- **Scope di questo repository = gestionale (MAUI) + DB.** Il sito Next.js (Fase 3) e la logica pagamenti Stripe + integrazione contabile (Fase 4) sono **binari separati**, pianificati a parte.
- **1° rilascio = Contenuti + Newsletter.** Pagamenti online e blog entrano come **sole tabelle di predisposizione** (create ma non cablate), da attivare in un secondo momento.
- Costruzione **sequenziale per Fasi**; piano esecutivo di dettaglio in `2026-07-03-Piano_Operativo_Estensione_Web_design.md` (13 blocchi) e `2026-07-03-Impl_Blocco0-3_Fondamenta_DB.md`.

B. Sicurezza / RLS — scoperta architetturale

- Le **RLS esistenti sono role-based native** (ruoli Postgres `app_*`), **non** usano `auth.*` di Supabase; il contesto tenant passa dal GUC `my.app_user` (`set_config`).
- Il **gestionale si connette come superuser postgres** → **bypassa le RLS**: l'isolamento multi-tenant reale vive **nelle funzioni PL/pgSQL**. Le **RLS diventano il confine di sicurezza SOLO per la lettura pubblica del sito** (ruolo `anon`, non-superuser).
- Conseguenza operativa: le policy pubbliche si scrivono **TO anon role-based** (niente `auth.*`) e sono **testabili in locale** con `CREATE ROLE anon + SET ROLE anon`, **senza Supabase CLI**. **FORCE RLS** non serve (l'app è superuser; `anon` non è owner).
- **Hardening EXECUTE**: `anon` eredita da `PUBLIC` l'`EXECUTE` su tutte le funzioni (incluse `SECURITY DEFINER` che bypassano RLS) → prima di esporre `anon` si esegue `REVOKE EXECUTE ON ALL FUNCTIONS IN SCHEMA public FROM PUBLIC` e si

concede `EXECUTE` solo alle `fn_web_*` pubbliche.

C. Storage media (decisione C2 “A+”)

- Backend = **bucket Supabase di test reale** dietro un **seam `IWebMediaStorage`** (accesso via **HTTP REST + service key**, niente SDK).
- **`storage_path` = sorgente di verità**; l’URL pubblico si **ricompone a runtime dal base-URL d’ambiente** (nessun dato “sporco” dev/prod).
- **NO Supabase CLI** in locale: troppo invasiva sulla toolchain di deploy consolidata (`deploy_sql.sh` , 400+ script numerati). La `ServiceKey` service-role **non** andrà nel binario client (upload/delete server-side o chiave scoped).

D. Convenzioni DB fissate

- Nuove tabelle con `azienda_id` (non `azienda_fk` legacy) + coda audit + **trigger condiviso `trg_web_audit()`** + RLS `superadmin_bypass_all` .
- **Larghezza FK**: verso PK `web_*` (BIGINT identity) → `BIGINT` ; verso PK `ana_*` (INTEGER) → `INTEGER` .
- **Indice azienda** standalone **solo** se non esiste già una UNIQUE/indice che parte da `azienda_id` .
- Segreti (ESP/Stripe) cifrati in `JSONB` **riusando il pattern `password_enc`** di `ana_aziende_smtp` .

E. Correzioni emerse verificando il DB reale

- **`ana_fornitori` NON esiste**: `ana_clienti.controparte_fk` aggiunta **senza FK** (predisposizione); target reale = `ana_controparti(controparte_id)` , vincolo in Fase 4.
- **`web_pagamenti_transazioni.mov_transazione_fk` = `INTEGER`** con **FK reale** a `mov_transazioni(transazione_id)` (PK legacy `INTEGER`) e **UNIQUE** (idempotenza 1:1 incasso→contabilità).
- **`ana_tipo_viaggi.web_categoria_fk` = `BIGINT`** (allineata alla PK identity).
- **CHECK lingua IN ('IT','FR','EN','DE','ES')** su newsletter/reminder; **ON DELETE SET NULL** sulle FK di arricchimento nullable.

F. Approccio “lean” ai componenti

- **Douglas-Peucker implementato a mano** (~30 righe, no NetTopologySuite); **Claude API** e **Supabase Storage** via `HttpClient` /REST (nessun SDK aggiuntivo). Presenti e riusati: SkiaSharp, ImageSharp, Blazored.TextEditor, QuestPDF, Npgsql.

G. Stato avanzamento (2026-07-03)

- **Blocco 0** (preparazione: ruolo `anon` , `trg_web_audit` , seam storage) e **Blocco 1** (19 tabelle + alter + rollback) **completati e verificati in locale**; **Blocco 2** (funzioni PL/pgSQL) in corso. Riferimenti campo-per-campo e diagramma E/R in `Dettaglio_Tabelle_DB.md` .